Continuous Compliance Monitoring & Audit-Evidence Automation (CIS/STIG on Intune + Entra)

Implementation Solution Guide

Version 1.0.0 · 2026-07-01

Contents

Executive Summary — Continuous Compliance Monitoring & Audit-Evidence Automation 4

The problem . . . 4

The solution . . . 4

What is included . . . 4

Business value . . . 4

Bottom line . . . 5

Solution Overview — Continuous Compliance Monitoring & Audit-Evidence Automation (CIS/STIG on Intune + Entra) 5

What this solution is . . . 5

The continuous-compliance model . . . 5

How it complements a one-time CIS/STIG build . . . 6

Where the evidence comes from . . . 6

Deployment model . . . 7

What you get . . . 7

Requirements — Continuous Compliance Monitoring & Audit-Evidence Automation 7

R1. An existing CIS/STIG baseline deployed to Intune + Entra . . . 7

R2. Intune + Entra roles with Microsoft Graph read access . . . 8

R3. A reporting / evidence destination . . . 8

R4. Defined evidence-retention needs . . . 9

R5. A defined pilot scope . . . 9

Implementation Guide — Continuous Compliance Monitoring & Audit-Evidence Automation 9

Stage 0 — Prepare (once) . . . 9

Stage 1 — Assess: define the monitored control set (the baseline reference) . . . 10

Stage 2 — Configure: set scope and measurement . . . 10

Stage 3 — Pilot: measure, detect drift, export evidence . . . 11

Stage 4 — Validate: confirm outputs meet audit requirements . . . 11

Stage 5 — Expand: fleet-wide and scheduled . . . 12

Script summary (order of use) . . . 12

Testing — Continuous Compliance Monitoring & Audit-Evidence Automation 12

T1 — Connectivity & scope (read-only) . . . 13

T2 — Baseline reference is complete . . . 13

T3 — Control measured (current vs. baseline) . . . 13

T4 — Drift detected . . . 13

T5 — Evidence exported (audit-ready) . . . 13

T6 — Remediation closes the gap (preview → commit → re-measure) . . . 14

T7 — Scheduled fleet cadence . . . 14
Troubleshooting — Continuous Compliance Monitoring & Audit-Evidence Automation 14
Connectivity & permissions . . . 14
Discovery (CCAUDIT-Discovery.ps1) . . . 15
Baseline reference . . . 15
Validation (CCAUDIT-Validation.ps1) . . . 15
Drift detection (Testing T4) . . . 16
Report / evidence (CCAUDIT-Report.ps1) . . . 16
Remediation (CCAUDIT-Remediation.ps1) . . . 16
Scope targeting (CCAUDIT-Set-ScopeTargeting.ps1) . . . 16
When to stop and escalate . . . 17
Best Practices — Continuous Compliance Monitoring & Audit-Evidence Automation 17
Evidence retention . . . 17
Cadence and drift handling . . . 18
Scoping and change control . . . 18
Audit response . . . 18
What to avoid . . . 19
Deliverables Summary — Continuous Compliance Monitoring & Audit-Evidence Automation 19
Documentation . . . 19
PowerShell scripts (Microsoft Graph SDK) . . . 20
Named objects you will create and reuse . . . 20
Outcome . . . 21
Scope note . . . 21
Appendix A — PowerShell Scripts 22

Executive Summary — Continuous Compliance Monitoring & Audit-Evidence Automation

The problem

Passing a CIS or STIG baseline **once** is not the same as **proving** you are still compliant months later. In a federally regulated environment, endpoints drift — a policy is edited, an exclusion is added, a device enrolls without the baseline — and when an auditor asks for evidence, teams scramble to gather it by hand. Point-in-time compliance and manual evidence collection are a standing audit risk.

The solution

A **repeatable operational method — with PowerShell scripts you keep** — to continuously monitor baseline **drift** and automate **audit evidence** for Intune + Entra endpoints. It follows a simple, provable cycle:

Baseline → Measure → Detect drift → Evidence / POA&M → Remediate / track.

The scripts read the live state of your CIS/STIG controls from Microsoft Graph (read-only), compare it to your approved baseline, flag every control that has drifted, and export **dated audit evidence** plus a **POA&M-ready gap list** — the exact artifacts an auditor expects. Drifted settings can then be safely re-applied (preview first) and re-measured to prove closure.

This is an **operations product**, distinct from a one-time baseline build. The build creates the policies; this solution proves they stay true and produces the paperwork on demand.

What is included

- A complete documentation set: Overview, Requirements, Implementation, Testing, Troubleshooting, Best

Practices, and Deliverables.

- **Five PowerShell scripts** (Microsoft Graph SDK): control-state discovery, per-device baseline

conformance, an audit-evidence + drift/gap report (POA&M-ready), safe remediation (preview by default), and a target-group builder (preview by default).

- A phased rollout — **Assess → Configure → Pilot → Validate → Expand** — with pass/fail gates and a

scheduled fleet-wide cadence.

Business value

- **Prove compliance any day**, not just deployment day.
- **Detect drift automatically** before an auditor does.
- **Cut audit-prep effort** — evidence and POA&M gaps export on demand, correctly formatted.
- **Reduce risk** — safe, preview-first remediation and full change control.

• **Own the capability** — a method and scripts you reuse across audit cycles, at a fraction of ongoing consulting cost.

Bottom line

Continuous, provable endpoint compliance — drift detected and audit evidence exported on demand — with a repeatable method and scripts you keep and reuse between audits. The specific CIS/STIG rule content is deferred to CIS and DISA; this solution delivers the precise monitoring and evidence method on Intune + Entra.

Solution Overview — Continuous Compliance Monitoring & Audit-Evidence Automation (CIS/STIG on Intune + Entra)

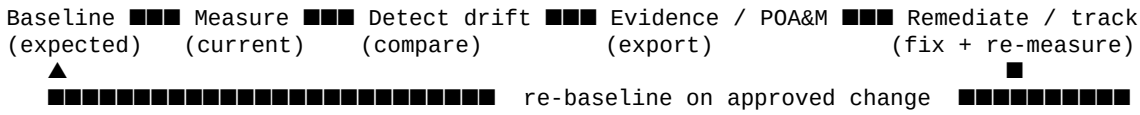
What this solution is

Passing a CIS or STIG baseline **once** is not the same as **proving** you are still compliant months later, when an auditor asks for evidence. Devices drift: a policy is edited, an exclusion is added, a new device enrolls without the baseline, a user with local rights changes a setting. This solution gives you a **repeatable, operational method** — with PowerShell scripts you keep and reuse — to:

1. **Baseline** — take the CIS/STIG control set you already deployed to Intune + Entra and treat it as the *expected state* (the reference you measure against).
1. **Measure** — read the *current state* of those controls for each in-scope device and identity directly from Microsoft Graph (read-only).
1. **Detect drift** — compare current state to the baseline and flag every control that no longer matches (drifted, missing, or newly non-conformant).
1. **Evidence / POA&M** — export the measurement as **audit evidence** (a dated, per-control, per-device record) and produce a **POA&M-ready gap list** (Plan of Action & Milestones) of the controls that fail, so remediation can be planned and tracked.
1. **Remediate / track** — safely re-apply the drifted settings (preview first) and re-measure to confirm the gap closed — then keep the cycle running on a schedule.

This is an **operations product**, distinct from a one-time baseline *build*. The build creates the policies; **this solution watches them stay true and produces the paperwork** an audit requires.

The continuous-compliance model



• **Baseline (expected state).** Your approved CIS/STIG configuration in Intune (Settings Catalog /

device-configuration / compliance policies / Security Baselines) and Entra (Conditional Access, authentication methods, tenant security settings). The specific CIS Benchmark / DISA STIG **rule IDs and official control text are deferred to CIS and DISA** — this solution does not reproduce or fabricate them. You supply *which* Intune/Entra objects encode your baseline; the method measures and evidences them.

- **Measure (current state).** The scripts query Microsoft Graph read-only for the live value of each monitored control on each in-scope device/identity — no changes are made during measurement.
- **Detect drift.** Current vs. expected, per control per device. Output is a conformance verdict: **Pass** (matches baseline), **Drift** (was compliant, now differs), **Missing** (baseline not applied to this device), or **Non-conformant** (never matched).
- **Evidence / POA&M.** Two exports: (a) an **audit-evidence** package — timestamped, immutable CSV/JSON showing what was measured, when, by whom, and the result; (b) a **POA&M-ready gap** list — each failing control with device count, severity, and a milestone/owner column ready to paste into your organization's POA&M template.
- **Remediate / track.** For drifted settings only, a safe remediation re-applies the baseline targeting (default **preview / -WhatIf** — nothing changes until you confirm). Re-measure to prove closure; the evidence trail records the fix.

How it complements a one-time CIS/STIG build

One-time baseline build	This continuous-compliance solution
Creates the CIS/STIG policies in Intune/Entra	Treats those policies as the measured reference
Proves compliance **on deployment day**	Proves compliance **on any day, on demand**
Output: configured tenant	Output: drift alerts + dated audit evidence + POA&M gaps
Runs once	Runs on a schedule (pilot, then fleet-wide)
Answers "did we set it?"	Answers "is it *still* set — and can we prove it?"

If you have not yet built a CIS/STIG baseline, do that first; this solution assumes an existing baseline to measure against (see Requirements). If you have, this is the operational layer that keeps it audit-defensible.

Where the evidence comes from

All evidence is derived from **authoritative, first-party sources** via Microsoft Graph — never hand-typed:

- **Intune configuration state** — device configuration / Settings Catalog profiles, compliance policies, and Security Baseline assignments and their reported values.
- **Intune device compliance & per-setting state** — each device's compliance status and, where exposed, per-setting reporting (`deviceCompliancePolicyDeviceStateSummary`, device configuration device-status).

• **Entra configuration state** — Conditional Access policies, authentication-method policy, and tenant security-relevant settings that encode identity controls in your baseline.

• **Directory / device inventory** — the in-scope device and group membership that defines *who the baseline should apply to*, so "Missing" can be distinguished from "Drift".

Each measurement record carries a **collection timestamp, the tenant, the operator identity, the control identifier, the expected value, the observed value, and the verdict** — the fields an auditor expects. Evidence is exported to a **reporting destination** you control (see Requirements) and retained per your **evidence-retention** policy.

Deployment model

This solution follows the category standard: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a **pass/fail gate** at each stage. You will first prove the method on a small **pilot group** (measure, detect a seeded drift, export evidence), validate the outputs against your audit requirements, then expand to the full fleet on a schedule. The scripts are read-only by default; the only script that changes anything (Remediation) and the targeting builder (Set-ScopeTargeting) run in **preview (-WhatIf)** until you explicitly commit.

What you get

• This documentation set (Overview, Requirements, Implementation, Testing, Troubleshooting, Best Practices, Deliverables).

• **Five PowerShell scripts** (Microsoft Graph SDK): control-state **Discovery**, per-device baseline **Validation**, an audit-evidence + drift/gap **Report** (POA&M-ready), safe **Remediation** of drifted settings (default preview), and a **Set-ScopeTargeting** pilot-group builder (default preview).

Requirements — Continuous Compliance Monitoring & Audit-Evidence Automation

Confirm each prerequisite below **before** you begin the Implementation Guide. Category-level prerequisites (Entra tenant, Intune subscription, admin roles, enrollment, pilot group, supported OS, network, change control) are listed once in **global-requirements-security.md** — read that first. The items here are **specific to continuous compliance monitoring and audit-evidence automation**. Each entry follows: **Description / Why / Verify / Implement if missing.**

R1. An existing CIS/STIG baseline deployed to Intune + Entra

Description: A CIS Benchmark and/or DISA STIG configuration already built and assigned in Intune (Settings Catalog / device-configuration / compliance policies / Security Baselines) and Entra (Conditional Access, authentication methods, tenant settings) — the *expected state* you will measure against.

Why: This solution measures **drift from a baseline**. Without a defined baseline there is nothing to compare current state to, no "expected value" for evidence, and no gap to put in a POA&M. The specific

CIS/STIG rule IDs and official control text come from **CIS and DISA** — this solution does not supply them.

Verify: In `https://intune.microsoft.com` → **Endpoint security** → **Security baselines** and/or **Devices** → **Manage devices** → **Configuration**, confirm your baseline profiles exist and are assigned. In `https://entra.microsoft.com` → **Protection** → **Conditional Access** confirm the identity controls exist. You should be able to name the exact objects that encode your baseline.

Implement if missing: Build the one-time CIS/STIG baseline first (a separate build activity): import the CIS/STIG content into Intune Settings Catalog / Security Baselines, create the matching Entra Conditional Access and authentication-method policies, and assign them. Then return here to monitor it.

R2. Intune + Entra roles with Microsoft Graph read access

Description: An operator identity holding read rights to Intune and Entra configuration and device state, and consented Microsoft Graph **read scopes** for the monitoring scripts.

Why: Discovery, Validation, and Report are **read-only** and must be able to read policies, compliance state, Conditional Access, and directory objects via Graph. Insufficient scopes cause empty or partial evidence — an audit gap.

Verify: In Entra → **Roles and administrators**, confirm the operator has at least a read role (for example **Global Reader** plus **Intune Administrator** for later remediation). Run `CCAUDIT-Discovery.ps1` (read-only) and confirm it connects and returns data without a permissions error.

Implement if missing: Assign **Global Reader** (directory/CA read) and **Intune Administrator** (Intune read/write for remediation). On first script run, consent the required delegated Graph read scopes when prompted: `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, `Policy.Read.All`, `Directory.Read.All`, `Group.Read.All`. (Remediation additionally needs `DeviceManagementConfiguration.ReadWrite.All`; Set-ScopeTargeting needs `Group.ReadWrite.All`.)

R3. A reporting / evidence destination

Description: A controlled location where the exported audit-evidence and POA&M-ready gap files are written and stored (for example a restricted, access-logged file share or document library the scripts' output folder points to).

Why: Audit evidence has value only if it is stored somewhere durable, access-controlled, and tamper-evident. Writing evidence to a random local folder undermines its integrity.

Verify: Confirm you have a path (local staging + a controlled destination) where the operator can write, that is backed up, access-restricted, and retained per policy. Confirm you can write a test file there.

Implement if missing: Create a dedicated folder / library (for example a SharePoint document library or restricted network share) with read/write limited to the compliance operators, versioning or immutability enabled where available, and backup configured. Point the scripts' `-OutputPath` at your local staging folder and copy finished evidence to the controlled destination.

R4. Defined evidence-retention needs

Description: A written statement of how long audit evidence must be kept, in what format, and who may access it — derived from your regulatory framework and audit cycle.

Why: Federally regulated environments mandate retention periods; the export cadence, storage, and disposal must match. Retention that is too short fails audits; unmanaged retention creates risk.

Verify: Confirm a retention requirement exists (period, format, access, disposal) and that your destination (R3) enforces it. Confirm ownership of the retention decision.

Implement if missing: Document the retention requirement with your compliance/audit lead (period, immutable vs. versioned, access list, disposal method). Configure the destination's retention/hold settings to match, and record the cadence (for example measure weekly, retain evidence per policy).

R5. A defined pilot scope

Description: A small, representative Microsoft Entra security group of devices (and/or users) used to prove the monitoring method before fleet-wide rollout — named `CCAUDIT-Pilot-Devices`.

Why: The category standard is Assess → Configure → **Pilot** → Validate → Expand. Piloting on a small group first lets you validate measurement accuracy, drift detection, and evidence output without noise or risk across the whole fleet, and lets you safely seed a test drift.

Verify: In Entra → **Groups**, confirm `CCAUDIT-Pilot-Devices` exists and contains a small set of representative in-scope devices (mix of OS builds / roles).

Implement if missing: Create it — Entra → **Groups** → **New group** → type **Security**, name `CCAUDIT-Pilot-Devices`, add a handful of representative enrolled devices — or generate it from your baseline scope with `CCAUDIT-Set-ScopeTargeting.ps1` (default preview / -WhatIf; review, then commit).

When R1–R5 and the global requirements are all satisfied, proceed to the Implementation Guide.

Implementation Guide — Continuous Compliance Monitoring & Audit-Evidence Automation

This guide gives you the **exact, ordered steps** to stand up continuous compliance monitoring and audit-evidence automation for your CIS/STIG baseline on Intune + Entra. Follow the category deployment model: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, with a pass/fail gate at each stage. All discovery/measurement is **read-only**; the only changing actions (Remediation, Set-ScopeTargeting) default to **preview (-WhatIf)**.

Work through the stages in order. Do not skip the pilot.

Stage 0 — Prepare (once)

1. Confirm every item in **02-requirements.md** and **global-requirements-security.md** is satisfied.

2. Install the Microsoft Graph PowerShell SDK on the operator workstation (elevated PowerShell):

```
Install-Module Microsoft.Graph -Scope CurrentUser
```

1. Copy the five **CCAUDIT-*.ps1** scripts to a working folder, for example

`C:\CCAUDIT\scripts`, and create an output folder `C:\CCAUDIT\out` (your local staging → copy finished evidence to the controlled destination from R3).

1. Sign in once to confirm scopes:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Discovery.ps1 -OutputPath C:\CCAUDIT\out
```

Consent the read scopes when prompted. **Gate 0 (pass):** the script connects and writes a discovery file with no permissions error.

Stage 1 — Assess: define the monitored control set (the baseline reference)

You are turning your deployed CIS/STIG baseline into a **machine-readable expected state** the scripts measure against.

1. **Discover current configuration.** Run control-state discovery to inventory the Intune and Entra

objects that encode your baseline:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Discovery.ps1 -OutputPath C:\CCAUDIT\out
```

It writes `CCAUDIT-Discovery-<timestamp>.csv/json` listing the discovered configuration profiles, compliance policies, Security Baselines, and Entra Conditional Access / authentication-method policies with their current settings.

1. **Select the monitored controls.** From the discovery output, choose the controls that make up

your monitored set — the CIS/STIG settings you must prove. Record, for each control: a stable **control identifier** (use your CIS/STIG reference verbatim — deferred to CIS/DISA), the **Intune or Entra object** that carries it, the setting name, and the **expected value**.

1. **Create the baseline reference file.** Save the selected controls as

`CCAUDIT-Baseline.json` in the working folder — this is your expected-state reference. Use the template the Discovery script emits (`-EmitBaselineTemplate`) so field names match what Validation reads. Store it under change control (R1/G8).

Gate 1 (pass): `CCAUDIT-Baseline.json` exists, each entry has a control ID, target object, setting, and expected value, and it is approved by your compliance lead.

Stage 2 — Configure: set scope and measurement

1. **Build the pilot target group.** If `CCAUDIT-Pilot-Devices` does not exist, create it —

preview first, then commit:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Set-ScopeTargeting.ps1 -GroupName CCAUDIT-Pilot-Devi
```

Review the previewed membership (default -WhatIf). Re-run with `-Commit` to create/populate it.

1. **Dry-run measurement against the baseline.** Validate current vs. expected for the pilot group:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Validation.ps1 -BaselinePath .\CCAUDIT-Baseline.json
```

It writes a per-device, per-control conformance file with verdicts (Pass / Drift / Missing / Non-conformant).

Gate 2 (pass): every monitored control is measured for every pilot device (no "not measured" rows caused by scope or scope-tag gaps).

Stage 3 — Pilot: measure, detect drift, export evidence

1. **Baseline measurement.** Run Validation against the pilot (as above) and keep the output as the

pilot **baseline measurement**.

1. **Seed a controlled drift (test).** In a change window, deliberately change one non-critical

monitored setting on one pilot device / one pilot-scoped policy (record the change). This proves detection works. Do **not** seed authentication-impacting drift.

1. **Detect the drift.** Re-run Validation. Confirm the seeded change appears as **Drift** for the

expected control/device, and unchanged controls remain **Pass**.

1. **Export audit evidence + POA&M gaps.** Generate the evidence and gap exports:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Report.ps1 -ValidationPath C:\CCAUDIT\out\CCAUDIT-Va
```

It writes: `CCAUDIT-Evidence-<timestamp>.csv/json` (dated per-control per-device evidence with tenant, operator, expected, observed, verdict) and `CCAUDIT-POAM-<timestamp>.csv` (one row per failing control with device count, severity, and empty milestone/owner columns to complete).

1. **Remediate the seeded drift (preview then commit).** Preview the fix, then commit:

```
powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Remediation.ps1 -ValidationPath C:\CCAUDIT\out\CCAUD
```

Default is -WhatIf (preview). Review the planned re-application of baseline targeting for the drifted control, then re-run with `-Commit`. Re-run Validation to confirm the control returns to **Pass**.

Gate 3 (pass): the seeded drift was detected, evidence exported with correct fields, POA&M row generated, and remediation returned the control to Pass on re-measure. Copy the evidence to the controlled destination (R3).

Stage 4 — Validate: confirm outputs meet audit requirements

1. Have your compliance/audit lead review a sample evidence export and confirm the fields, format, and

retention match audit expectations (R4).

1. Confirm the POA&M export maps cleanly into your organization's POA&M template (control ID, weakness

description, affected devices, severity, milestones, owner).

1. Confirm evidence lands in the controlled destination and is retained per policy.

Gate 4 (pass): audit lead accepts the evidence and POA&M formats; retention confirmed.

Stage 5 — Expand: fleet-wide and scheduled

1. **Broaden scope.** Point Validation/Report at your full in-scope set (a fleet group such as `CCAUDIT-Fleet-Devices`, built with Set-ScopeTargeting) instead of the pilot group.
1. **Schedule the cycle.** Create a scheduled task on the operator workstation (or a managed runbook host) that runs, on your chosen cadence (for example weekly): Discovery → Validation → Report, writing to staging and copying evidence to the controlled destination.
1. **Route drift.** Treat each new **Drift** / **Missing** / **Non-conformant** row as an event: open a POA&M item, remediate under change control (Remediation preview → commit), and re-measure to close.
1. **Re-baseline on approved change.** When your CIS/STIG baseline legitimately changes (approved), re-run Discovery, update `CCAUDIT-Baseline.json`, and re-approve it — so expected state stays accurate.

Gate 5 (pass): the scheduled cycle runs unattended, evidence accumulates in the controlled destination on cadence, and drift is tracked to closure via POA&M.

Script summary (order of use)

Order	Script	Purpose	Changes anything?
1	`CCAUDIT-Discovery.ps1`	Inventory control state; emit baseline template	No (read-only)
2	`CCAUDIT-Set-ScopeTargeting.ps1`	Build pilot/fleet target groups	Preview by default (-WhatIf)
3	`CCAUDIT-Validation.ps1`	Per-device baseline conformance (drift verdicts)	No (read-only)
4	`CCAUDIT-Report.ps1`	Audit evidence + POA&M-ready gap export	No (read-only)
5	`CCAUDIT-Remediation.ps1`	Safe fix of drifted settings	Preview by default (-WhatIf)

Proceed to **05-testing.md** for the formal pass/fail gates and **06-troubleshooting.md** if a step does not behave as described.

Testing — Continuous Compliance Monitoring & Audit-Evidence Automation

Use these **pass/fail gates** to prove the solution works before you rely on it for an audit. The three core outcomes you must demonstrate are: **a control is measured**, **drift is detected**, and **evidence is exported**. Run the gates on the pilot group (`CCAUDIT-Pilot-Devices`) first, then confirm the fleet cadence.

Record the result of each gate (Pass / Fail), the date, and the operator. Do not expand to the fleet until Gates T1–T6 pass.

T1 — Connectivity & scope (read-only)

Test: Run `CCAUDIT-Discovery.ps1 -OutputPath C:\CCAUDIT\out`. **Pass:** Script connects to Microsoft Graph with the read scopes, completes without a permissions error, and writes `CCAUDIT-Discovery-<timestamp>.csv/json` containing your baseline's Intune profiles, compliance policies, Security Baselines, and Entra Conditional Access / auth-method policies. **Fail:** Permissions error, empty output, or missing baseline objects → see R2 and Troubleshooting.

T2 — Baseline reference is complete

Test: Open `CCAUDIT-Baseline.json`. **Pass:** Every monitored control has a stable **control ID**, a **target object** (Intune/Entra), the **setting name**, and an **expected value**; the file is approved under change control. **Fail:** Any control missing an expected value or target object → re-run Discovery with `-EmitBaselineTemplate` and complete the entry.

T3 — Control measured (current vs. baseline)

Test: Run `CCAUDIT-Validation.ps1 -BaselinePath .\CCAUDIT-Baseline.json -GroupName CCAUDIT-Pilot-Devices -OutputPath C:\CCAUDIT\out`. **Pass:** Every monitored control is measured for **every** pilot device — each row has a verdict of **Pass / Drift / Missing / Non-conformant** and an observed value. No "not measured" rows. **Fail:** Controls or devices missing from the output, or verdicts blank → check group membership, scope tags, and Graph read scopes (R2, R5, Troubleshooting).

T4 — Drift detected

Test: In a change window, deliberately change one non-critical monitored setting on one pilot device or pilot-scoped policy (record it). Re-run `CCAUDIT-Validation.ps1`. **Pass:** The changed control shows **Drift** for the expected device with the new observed value; all untouched controls remain **Pass**. Detection is specific (only the seeded control flips). **Fail:** The change is not detected, or unrelated controls flip → verify you edited a *monitored* control and that Validation reads the same object; see Troubleshooting.

T5 — Evidence exported (audit-ready)

Test: Run `CCAUDIT-Report.ps1 -ValidationPath <latest validation json> -OutputPath C:\CCAUDIT\out`. **Pass:** `CCAUDIT-Evidence-<timestamp>.csv/json` is written

and each record contains **collection timestamp, tenant, operator identity, control ID, target object, expected value, observed value, and verdict**. `CCAUDIT-POAM-<timestamp>.csv` is written with **one row per failing control** including device count, severity, and empty **milestone/owner** columns. Files copy successfully to the controlled destination (R3). **Fail:** Missing fields, no POA&M rows for known failures, or destination write fails → check R3/R4 and Troubleshooting.

T6 — Remediation closes the gap (preview → commit → re-measure)

Test: Run `CCAUDIT-Remediation.ps1 -ValidationPath <latest validation json>` (default -WhatIf) and review the preview; re-run with `-Commit`; then re-run `CCAUDIT-Validation.ps1`. **Pass:** Preview lists the exact drifted control to be re-applied and changes nothing; after `-Commit`, re-measurement shows the control back to **Pass**; the evidence trail records the fix. **Fail:** Preview changes state, commit does not close the gap, or re-measure still shows Drift → do not use Remediation on the fleet; investigate per Troubleshooting.

T7 — Scheduled fleet cadence

Test: After Gates T1–T6 pass, point Validation/Report at the fleet group (`CCAUDIT-Fleet-Devices`) and run the scheduled Discovery → Validation → Report cycle once unattended. **Pass:** The cycle completes on schedule, evidence lands in the controlled destination, and any drift appears as POA&M rows. **Fail:** Task does not run unattended, or evidence does not accumulate → check the scheduled-task account's Graph consent and destination permissions.

Overall pass criteria: T1–T6 pass on the pilot and T7 passes on the fleet. Only then treat the solution as production audit evidence.

Troubleshooting — Continuous Compliance Monitoring & Audit-Evidence Automation

Use this guide when a step in the Implementation or Testing guide does not behave as described. Each entry is **Symptom** → **Likely cause** → **Fix.** Items map to the scripts and pass/fail gates.

Connectivity & permissions

Symptom: A script fails on connect with a consent or permissions error, or returns no data. **Likely cause:** The operator lacks a required role, or the delegated Graph read scopes were not consented. **Fix:** Confirm the operator holds at least **Global Reader** plus **Intune Administrator** (R2, G3). Re-run and consent the read scopes when prompted: `DeviceManagementConfiguration.Read.All`, `DeviceManagementManagedDevices.Read.All`, `Policy.Read.All`, `Directory.Read.All`, `Group.Read.All`. If a tenant admin must grant consent, have them approve the app's delegated permissions, then re-run.

Symptom: `Microsoft.Graph module not found`. **Likely cause:** The Graph PowerShell SDK is not installed. **Fix:** `Install-Module Microsoft.Graph -Scope CurrentUser`, then re-run.

Symptom: Scheduled fleet run works interactively but not unattended (T7). **Likely cause:** The scheduled-task account has not consented the Graph scopes, or cannot write to the destination. **Fix:** Run each script once interactively as the task account to complete consent; grant that account write access to the staging/destination folders (R3).

Discovery (CCAUDIT-Discovery.ps1)

Symptom: Discovery runs but your baseline profiles/policies are missing from the output. **Likely cause:** The baseline was never built/assigned, or lives in an object type not queried. **Fix:** Confirm the baseline exists in Intune (Endpoint security → Security baselines; Devices → Configuration) and Entra (Conditional Access) — see R1. If custom objects are used, note their type; you can still record them manually in `CCAUDIT-Baseline.json`.

Symptom: Baseline template has placeholder values (`REPLACE-...`). **Likely cause:** The template is meant to be completed by you. **Fix:** Fill in each control's **ControlId** (your CIS/STIG reference — deferred to CIS/DISA), **SettingName**, **ExpectedValue**, and **Severity**, save as `CCAUDIT-Baseline.json`, and approve it under change control (Gate 1/T2).

Baseline reference

Symptom: Validation throws "Baseline reference not found" or "is empty". **Likely cause:** `CCAUDIT-Baseline.json` is missing, mis-pathed, or invalid JSON. **Fix:** Point `-BaselinePath` at the correct file; validate the JSON parses (`Get-Content ... | ConvertFrom-Json`). Regenerate from Discovery `-EmitBaselineTemplate` if needed.

Validation (CCAUDIT-Validation.ps1)

Symptom: "Target group not found." **Likely cause:** `CCAUDIT-Pilot-Devices` (or the named group) does not exist. **Fix:** Create it via Entra → Groups, or run `CCAUDIT-Set-ScopeTargeting.ps1` (preview then `-Commit`). Confirm the display name matches exactly (R5).

Symptom: No device rows / "No device members found." **Likely cause:** The target group has no device members, or contains users instead of devices. **Fix:** Add the representative **enrolled devices** to the group. For device-based measurement, the group members must be devices with an `azureADDeviceId`.

Symptom: Controls show verdict **Missing** for devices you know are compliant. **Likely cause:** The device is not returned in managed-device state (not enrolled/reporting), or the baseline object is not assigned to that device. **Fix:** Confirm the device is enrolled and compliant in Intune → Devices → All devices (G4). Confirm the baseline is assigned to a group that includes the device. "Missing" is a targeting/enrollment signal, not a drift.

Symptom: Every control shows the same verdict for a device regardless of setting. **Likely cause:** Observed value is derived from overall device compliance where per-setting reporting is unavailable, so all controls share the device's compliance signal. **Fix:** This is expected where Intune does not expose per-setting state for that object type; refine `CCAUDIT-Baseline.json` to point each control at an object whose state is individually reportable, or accept device-level conformance as the observed signal for that control.

Drift detection (Testing T4)

Symptom: A deliberately-changed setting is **not** detected as Drift. **Likely cause:** You changed a setting that is not in the monitored set, or Validation reads a different object than the one you edited. **Fix:** Confirm the changed setting corresponds to a control in `CCAUDIT-Baseline.json` and that its **TargetObject** is the object you edited. Allow time for the device to report the new state, then re-run.

Symptom: Unrelated controls flip to Drift after one change. **Likely cause:** Shared observed-value signal (device compliance) affecting multiple controls, or a broad policy change. **Fix:** Narrow the baseline entries to individually-reportable objects; verify only the intended policy was changed.

Report / evidence (CCAUDIT-Report.ps1)

Symptom: "Validation file not found" or "is empty." **Likely cause:** No validation run was produced, or the wrong path/timestamp was passed. **Fix:** Run `CCAUDIT-Validation.ps1` first; pass the exact `CCAUDIT-Validation-<timestamp>.json`.

Symptom: Evidence rows are missing provenance fields (T5). **Likely cause:** The validation output was hand-edited or truncated. **Fix:** Do not edit validation output. Re-run Validation to regenerate a complete file, then re-report.

Symptom: No POA&M rows despite known failures. **Likely cause:** No rows carry Drift/Missing/Non-conformant verdicts (all Pass), or the validation file is stale. **Fix:** Confirm failures exist in the validation output; re-run Validation against current state.

Symptom: Evidence cannot be copied to the destination. **Likely cause:** Missing write permission or the destination path is wrong (R3). **Fix:** Grant the operator write access; verify the destination path; confirm retention/immutability settings (R4) do not block the write path used for new files.

Remediation (CCAUDIT-Remediation.ps1)

Symptom: Preview appears to change state. **Likely cause:** `-Commit` was passed (which disables preview). **Fix:** Run without `-Commit` for preview. Only add `-Commit` inside an approved change window with a documented rollback (G8).

Symptom: After `-Commit`, the control does not return to Pass on re-measure. **Likely cause:** The re-assignment did not reach the device yet, or the drift source persists (for example a local admin re-changing the setting). **Fix:** Allow policy sync time and re-run Validation. If it recurs, address the root cause (remove local rights, correct the policy) rather than repeatedly remediating.

Symptom: "Baseline group not found." **Likely cause:** `CCAUDIT-Fleet-Devices` (or the named baseline group) does not exist. **Fix:** Build it with `CCAUDIT-Set-ScopeTargeting.ps1` (preview then `-Commit`).

Scope targeting (CCAUDIT-Set-ScopeTargeting.ps1)

Symptom: Group/members not created after `-Commit`. **Likely cause:** The operator lacks `Group.ReadWrite.All`, or candidate objects were not devices. **Fix:** Consent `Group.ReadWrite.All`; confirm candidates are device objects. Re-run preview to verify the plan, then `-Commit`.

Symptom: Pilot group has too many/few members. **Likely cause:** `-PilotSampleSize` default, or a source group with unexpected membership. **Fix:** Set `-PilotSampleSize` to the desired count, or pass `-SourceGroupName` to seed from a known group; review the preview plan before committing.

When to stop and escalate

- Do **not** run Remediation on the fleet if the pilot gates (T4/T6) did not pass.
- Do **not** treat exports as audit evidence until T1–T6 pass on the pilot and T7 on the fleet.
- If evidence integrity is ever in doubt (edited files, gaps in the series), re-run the full cycle to regenerate authoritative records rather than patching output by hand.

Best Practices — Continuous Compliance Monitoring & Audit-Evidence Automation

These practices keep your continuous-compliance program **defensible, low-risk, and audit-ready** over time. They focus on the two areas auditors and regulators scrutinize most: **evidence retention** and **audit response**.

Evidence retention

- **Retain the raw and the report.** Keep both the per-device Validation output *and* the finished Evidence/POA&M exports. The raw measurement is the primary record; the report is the derived artifact. An auditor may ask to see the underlying data.
- **Make evidence immutable where possible.** Store exports in a versioned or write-once (immutable / hold-enabled) location. If the destination cannot enforce immutability, restrict write access to the compliance operators and enable file versioning and backup.
- **Match retention to your framework, not convenience.** Set the retention period from your regulatory requirement and audit cycle (R4), not from disk space. When in doubt, keep longer.
- **Keep the full provenance chain.** Every export already carries collection timestamp, tenant, operator, control ID, expected value, observed value, and verdict — do not strip these fields when archiving or converting formats. They are what make the record credible.
- **Never store secrets in evidence.** Evidence is configuration state, not credentials. Do not place tokens, keys, or connection secrets in the output folder or the exports.
- **Timestamp and name consistently.** Keep the `CCAUDIT-<type>-<timestamp>` naming so files sort

chronologically and map to a run. Do not overwrite prior exports — accumulate them.

- **Back up the baseline reference too.** `CCAUDIT-Baseline.json` is the expected state your evidence

is measured against; version it under change control alongside the evidence so a past verdict can be reproduced.

Cadence and drift handling

- **Measure on a fixed schedule** (for example weekly) plus **after any approved baseline change** —

re-baseline immediately so expected state stays accurate and does not generate false drift.

- **Treat every Drift/Missing/Non-conformant row as an event.** Open a POA&M item, remediate under

change control, and re-measure to close. Do not let drift rows accumulate unactioned.

- **Distinguish Missing from Drift.** "Missing" (baseline never applied to a newly enrolled device)

is usually a targeting/enrollment fix; "Drift" (was compliant, changed) is a remediation. Handle them differently.

- **Remediate safely.** Always run Remediation in preview (-WhatIf) first, review the exact controls,

and only `-Commit` inside a change window with a rollback recorded (G8). Never bulk-remediate authentication-impacting controls without a pilot.

Scoping and change control

- **Pilot every change.** Validate the method on `CCAUDIT-Pilot-Devices` before the fleet; validate

any baseline change on the pilot before re-baselining fleet-wide.

- **Keep scope explicit.** Drive measurement from named groups (`CCAUDIT-Pilot-Devices`,

`CCAUDIT-Fleet-Devices`) so "who the baseline applies to" is unambiguous and Missing is meaningful.

- **Least privilege for the operator.** Read scopes for measurement; grant write scopes only for the

remediation identity, and only when remediating.

Audit response

- **Have a one-command evidence path.** When an auditor asks "prove control X is compliant across the

fleet on date Y," you should be able to produce it from the retained exports (or a fresh run) without manual data gathering.

- **Lead with the current export, support with the trail.** Present the latest Evidence export as the

point-in-time proof and the accumulated series as proof of *continuous* compliance over the period.

- **Map POA&M rows to remediation records.** For any failing control, show the POA&M item, the

remediation change ticket, and the re-measurement that closed it — this demonstrates a working control process, which is often what the audit is really assessing.

• **Show the schedule.** Evidence that the measurement runs on a defined cadence (the scheduled task and its history) is itself audit evidence of continuous monitoring.

• **Keep the method documented and reproducible.** Retain this documentation set and the script versions used so a past result can be reproduced and the process explained to an assessor.

• **Rotate and review access.** Periodically review who can read the evidence destination and who holds the remediation write role; remove stale access.

What to avoid

• Do not hand-edit evidence exports — it destroys their credibility. Re-run instead.

• Do not measure against an unapproved or stale baseline — verdicts become meaningless.

• Do not expose the output folder broadly — evidence is sensitive configuration state.

• Do not skip re-measurement after remediation — closure must be proven, not assumed.

Deliverables Summary — Continuous Compliance Monitoring & Audit-Evidence Automation

This package gives you everything needed to **monitor CIS/STIG baseline drift and automate audit evidence** for Intune + Entra endpoints — a repeatable operations method you keep and reuse between audits. Below is exactly what is included and what each piece is for.

Documentation

Document	What it gives you
01-solution-overview.md	The continuous-compliance model (baseline → measure → detect drift → evidence/POA&M → remediate/track), how it complements a one-time CIS/STIG build, and where evidence comes from.
02-requirements.md	Every prerequisite (existing baseline, Intune/Entra roles + Graph read access, reporting destination, evidence-retention needs, pilot scope) with Verify / Implement-if-missing.
global-requirements-security.md	Category-level prerequisites shared by all security solutions (tenant, Intune, roles, enrollment, pilot group, OS, network, change control).
03-implementation.md	Exact ordered steps: define the monitored control set, measure current vs. baseline, schedule drift detection, and generate audit-evidence + POA&M-ready gap exports — pilot then fleet-wide, with pass/fail gates.

Document	What it gives you
05-testing.md	Formal pass/fail gates: control measured, drift detected, evidence exported, remediation closes the gap, scheduled cadence.
06-troubleshooting.md	Symptom → cause → fix for the common issues (connectivity/scopes, empty measurement, missed drift, export/retention, remediation).
07-best-practices.md	Evidence retention and audit-response practices for a defensible program.
08-deliverables-summary.md	This file.
executive-summary.md	One-page business summary for decision-makers.

PowerShell scripts (Microsoft Graph SDK)

All discovery/measurement scripts are **read-only**. The two scripts that can change state default to **preview (-WhatIf)** and require an explicit `-Commit` to act.

Script	Purpose	Changes state?
CCAUDIT-Discovery.ps1	Inventory the control state of your baseline (Intune profiles, compliance policies, Security Baselines, Entra CA / auth-method policies); can emit a baseline-reference template.	No — read-only
CCAUDIT-Validation.ps1	Measure per-device conformance of current state vs. the baseline reference; verdicts: Pass / Drift / Missing / Non-conformant.	No — read-only
CCAUDIT-Report.ps1	Produce the dated audit-evidence export and the POA&M-ready gap export from a validation run.	No — read-only
CCAUDIT-Remediation.ps1	Safely re-apply drifted settings to return controls to baseline.	Preview by default (-WhatIf); `-Commit` to act
CCAUDIT-Set-ScopeTargeting.ps1	Build/populate the pilot and fleet target groups (`CCAUDIT-Pilot-Devices`, `CCAUDIT-Fleet-Devices`).	Preview by default (-WhatIf); `-Commit` to act

Named objects you will create and reuse

- `CCAUDIT-Pilot-Devices` — Entra security group for the pilot scope.
- `CCAUDIT-Fleet-Devices` — Entra security group for fleet-wide scope.

- `CCAUDIT-Baseline.json` — the approved expected-state reference measured against.
- **Evidence exports** — `CCAUDIT-Evidence-<timestamp>.csv/json`.
- **POA&M exports** — `CCAUDIT-POAM-<timestamp>.csv`.

Outcome

When deployed, you can prove endpoint compliance **on any day**, not just deployment day: drift is detected automatically, audit evidence is exported on demand, and gaps arrive POA&M-ready — with a method and scripts you own and reuse across audit cycles.

Scope note

The specific **CIS Benchmark and DISA STIG rule IDs and official control text are deferred to CIS and DISA**. This solution supplies the precise **monitoring and evidence method** on Intune + Entra; you supply which of your deployed baseline objects encode each control.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CCAUDIT-Discovery.ps1

Purpose: Discover and inventory the Intune and Entra objects that encode your deployed CIS/STIG baseline: device-configuration and Settings Catalog profiles, compliance policies, Security Baselines, and Entra Conditional Access and authentication-method policies. Optionally emits a baseline-reference template (CCAUDIT-Baseline.json) so you can record the expected value for each monitored control. Read-only — nothing is changed. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Discovery.ps1 -OutputPath C:\CCAUDIT\out

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Discovery.ps1 -OutputPath C:\CCAUDIT\out

CCAUDIT-Remediation.ps1

Purpose: For controls a validation run flagged as Drift, safely re-apply the baseline by (re)assigning the baseline target group to the affected policy so drifted devices return to expected state. Runs in preview (-WhatIf) by default: it lists exactly what it would do and changes nothing. Nothing is modified until you re-run with -Commit inside a change window. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Remediation.ps1 -ValidationPath C:\CCAUDIT\out\CCAUDIT-Validation-20260701-120000.json -OutputPath C:\CCAUDIT\out (add -Commit to apply)

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Remediation.ps1 -ValidationPath C:\CCAUDIT\out\CCAUDIT-Validation-20260701-120000.json -OutputPath C:\CCAUDIT\out (add -Commit to apply)

CCAUDIT-Report.ps1

Purpose: Turn a validation run into two auditor-ready exports: an audit-evidence package (dated, per-control, per-device record with tenant, operator, expected, observed, and verdict) and a POA&M-ready gap list (one row per failing control with affected-device count, severity, and empty milestone/owner columns to complete). Read-only — it consumes an existing validation file and writes exports; it does not query for changes or modify anything. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Report.ps1 -ValidationPath C:\CCAUDIT\out\CCAUDIT-Validation-20260701-120000.json -OutputPath C:\CCAUDIT\out

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Report.ps1 -ValidationPath C:\CCAUDIT\out\CCAUDIT-Validation-20260701-120000.json -OutputPath C:\CCAUDIT\out

CCAUDIT-Set-ScopeTargeting.ps1

Purpose: Create and populate the Entra security groups that define monitoring scope — the pilot group (CCAUDIT-Pilot-Devices) and the fleet group (CCAUDIT-Fleet-Devices). Runs in preview (-WhatIf) by default: it shows the group it would create and the members it would add, and changes nothing. Nothing is created or modified until you re-run with -Commit. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Set-ScopeTargeting.ps1 -GroupName CCAUDIT-Pilot-Devices -OutputPath C:\CCAUDIT\out

(add -Commit to create/populate)

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Set-ScopeTargeting.ps1 -GroupName CCAUDIT-Pilot-Devices -OutputPath C:\CCAUDIT\out (add -Commit to create/populate)

CCAUDIT-Validation.ps1

Purpose: Measure the current state of each monitored control against the approved baseline reference (CCAUDIT-Baseline.json) for every device in a target group, and assign a conformance verdict per device per control: Pass, Drift, Missing, or Non-conformant. This is how baseline drift is detected. Read-only — nothing is changed. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Validation.ps1 -BaselinePath .\CCAUDIT-Baseline.json -GroupName CCAUDIT-Pilot-Devices -OutputPath C:\CCAUDIT\out

How to run: powershell -ExecutionPolicy Bypass -File .\CCAUDIT-Validation.ps1 -BaselinePath .\CCAUDIT-Baseline.json -GroupName CCAUDIT-Pilot-Devices -OutputPath C:\CCAUDIT\out